Research Project Report

Part IV Project Final Report

Evaluating the Suitability of Blockchain Frameworks for New Zealand's General Election

Patrick Sheng

Co-worker: Eason Chang

Supervisor: Dr. Dev Singh

Co-Supervisor: Prof. Kevin Sowerby

Department of Electrical, Computer, and Software Engineering
The University of Auckland

18 October 2026

EVALUATING THE SUITABILITY OF BLOCKCHAIN FRAMEWORKS FOR NEW ZEALAND'S GENERAL ELECTION

Patrick Sheng

ABSTRACT

Online voting offers accessibility and cost advantages over paper ballots, but no proposed system has yet satisfied the security, privacy, and timeliness requirements of a national election simultaneously. Prior work in this department evaluated Minimal Anti-Collusion Infrastructure as a candidate and found it wanting on performance grounds, with vote confirmation on public Ethereum averaging 10.72 seconds and off-chain proof generation for five million votes extrapolating to approximately 2.86 weeks. The first bottleneck was attributed to public infrastructure never designed for an electoral workload, and the second to the cryptography used to prove the count. This project takes the opposite starting point and builds an electronic voting system directly on a private permissioned blockchain, using Hyperledger Besu with IBFT 2.0 as the sole consensus and ordering layer and no cryptographic voting protocol above it. Every validator checks each vote against a voting smart contract, which also keeps a running tally, and a vote is only recorded once a quorum of validators agrees on the block containing it, so that no single node can add, alter, or duplicate a vote. This report describes the network design, the consensus mechanism, how votes are recorded and tallied, and the configuration decisions that a private chain makes available and a public chain does not. Performance is measured across networks of four to sixteen validators against a worst-case target of 100 votes per second, alongside tests of validator failures and voter cheating attempts, and the key tests are repeated on an election-scale chain of 3.3 million registered voters and 1 million votes. The results are compared against the published public-network baselines. The system is shown to meet the throughput and latency requirements of the New Zealand context, at the cost of the receipt-freeness and collusion resistance that motivated the earlier work.

SIGNED STATEMENT OF CONTRIBUTION

Student

I hereby declare that:

1. This report is the result of the final year project work carried out by my project partner (see cover page) and I under the guidance of our supervisor (see cover page) in the 2026 academic year at the Department of Electrical, Computer, and Software Engineering, Faculty of Engineering and Design, University of Auckland.
2. This report is not the outcome of work done previously.
3. This report is not the outcome of work done in collaboration, except that with a potential project sponsor (if any) as stated in the text.
4. This report is not the same as any report, thesis, conference article or journal paper, or any other publication or unpublished work in any format.

In the case of a continuing project, please state clearly what has been developed during the project and what was available from previous year(s):

The 2025 project benchmarked e-voting on public Ethereum infrastructure, and its results were used here as a baseline only. This year we designed and built a private permissioned network using Hyperledger Besu with IBFT 2.0 consensus, along with the voting smart contract, the frontend, and all benchmarking and analysis.

Signature: 

Date: 17/10/2026

Supervisor

I confirm that the project work undertaken by this student in the 2026 academic year is / is ~~not~~ (strikethrough as appropriate) part of a continuing project, components of which have been completed previously. Comments, if any:

Signature: 

Date: 16/10/2026

Table of Contents

Acknowledgements . . . . . **vi**

Glossary of Terms . . . . . **vii**

Abbreviations . . . . . **viii**

1 Introduction **1**

2 Background and Literature Review **1**
2.1 Voting in New Zealand Today 1
2.2 Opportunities and Risks of Online Voting 1
2.3 Requirements for a National Online Voting System 2
2.4 Estimating New Zealand Election Load 2
2.5 Distributed Ledgers and Blockchain 3
2.5.1 Why a Decentralised Distributed System 3
2.5.2 How a Blockchain Works 4
2.5.3 Blocks and Block Period 4
2.5.4 Public and Permissioned Blockchains 4
2.6 Previous Evaluation of MACI 5
2.6.1 How MACI works 5
2.6.2 Performance Findings 5
2.6.3 Implications for This Project 6
2.7 Byzantine Fault Tolerant Consensus 6
2.7.1 The Agreement Problem 6
2.7.2 IBFT 2.0 Protocol 6
2.7.3 The Cost of Adding Validators 7
2.8 Hyperledger Besu 7

3 System Architecture and Design **8**
3.1 System Overview 8
3.2 Network Setup 9
3.3 Voting Smart Contract 9
3.4 How a Vote is Recorded 10
3.5 How Votes are Tallied 11
3.6 Consensus Rounds and Timing 11
3.7 Fault Tolerance 12
3.8 Scaling the Validator Set 13

4 Testing Methodology **14**
4.1 Overview 14
4.2 Test Environment 14
4.3 Benchmarks 15
4.3.1 Speed and Capacity 15
4.3.2 Network Size 16
4.3.3 Validator Failures 16
4.3.4 Cheating and Counting 16

5 Results and Analysis **17**
5.1 Speed and Capacity 17

5.2 Network Size 18
5.3 Validator Failures 20
5.4 Cheating and Counting 21

6 Discussion 22
6.1 Recommended Network Size 22
6.2 Comparison with MACI 23
6.3 Ballot Secrecy 24
6.4 Lessons for Deployment 24
6.5 Limitations 25

7 Conclusions and Future Work 26

References 27

Acknowledgements

I would like to thank our supervisor, Dr. Dev Singh, and co-supervisor, Prof. Kevin Sowerby, for their guidance and direction throughout this project. Our weekly meetings kept us on track and made for great discussions about our work.

I would also like to thank my project partner, Eason Chang, who was a pleasure to collaborate with. His consideration and dedication to the project made the work both easier and more enjoyable.

Thanks also go to Chris Kwon and Joshua Browne, whose evaluation of MACI laid the foundation for this project and provided the baseline for our results.

Finally, I would like to thank my family and friends for their support over the course of this year.

Glossary of Terms

Term	Definition
Block	A group of votes that the validators agree on and add to the blockchain together.
Block period	The minimum time between one block and the next, set to 2 seconds in this project.
Byzantine fault	A fault in which a node behaves arbitrarily, including sending contradictory information to different peers, rather than simply failing to respond.
Consensus protocol	The set of rules that nodes follow to agree on which block is added next.
Event	A log entry written to the blockchain by a smart contract. In this project, one is written for every recorded vote.
Finality	The point at which a committed block can no longer be reverted or reordered by any subsequent round of consensus.
Genesis file	The configuration document that defines the initial state of a blockchain, including the chain identifier, block parameters, initial validator set, and any pre-funded accounts.
Nonce	A sequence number carried by every transaction from an account. Each nonce can only be used once, which stops a copied transaction from being processed again.
Permissioned blockchain	A blockchain in which participation in consensus is restricted to a predefined and identified set of nodes rather than being open to anonymous participants.
Proposer	The validator responsible for assembling and broadcasting a candidate block for a given round of consensus.
Quorum	The minimum number of validators, about two-thirds of the network, that must approve a block before it is added.
Receipt-freeness	The property of a voting system that a voter cannot produce evidence of how they voted that would convince a third party.
Round change	The recovery procedure by which validators abandon a failed consensus round and begin a new one at the same block height.
Round timer	How long validators wait for a round to finish before starting a round change, set to 4 seconds in this project.
Smart contract	Code deployed to a blockchain that executes deterministically under predefined conditions.
Transaction pool	The holding area on each node for votes that have been submitted but not yet included in a block.
Validator	A node that participates in consensus by proposing blocks and voting on the proposals of others.

Abbreviations

Abbreviation	Meaning
CPU	Central Processing Unit
DLT	Distributed Ledger Technology
GCSB	Government Communications Security Bureau
IBFT	Istanbul Byzantine Fault Tolerance
MACI	Minimal Anti-Collusion Infrastructure
NZSIS	New Zealand Security Intelligence Service
PBFT	Practical Byzantine Fault Tolerance
p50, p95, p99	50th (median), 95th and 99th percentile latency

1. Introduction

New Zealand counts its general election votes by hand. Close to 2.9 million votes were cast in 2023 [1]. A preliminary result is reported on election night, but the official result, which includes special votes and a full recount, took 20 days in both the 2020 and 2023 elections [2]. The process is labour-intensive and slow, and it requires voters to physically reach a voting place or arrange a special vote in advance.

Online voting could make elections faster and more accessible, but a conventional online system stores every vote on servers run by a single organisation. Anyone who controls or breaks into those servers could change the result, and this risk is a large part of why New Zealand has not trialled online voting [3]. Blockchain technology offers an alternative. Votes are recorded on many computers run by different organisations, a vote is only accepted once most of them agree, and the tally is kept by the computers themselves and can be re-tallied independently (Section 3.5). Earlier research in this department evaluated MACI, a blockchain voting protocol built on the public Ethereum network, and found it too slow for a New Zealand general election, with votes taking several seconds to confirm and the final result taking weeks to certify [2, 4].

This project evaluates a different approach. Instead of adding a cryptographic protocol on top of a shared public network, we build a voting system directly on a private blockchain run by a small set of trusted, independent organisations, using Hyperledger Besu with the IBFT 2.0 consensus protocol. The research question is whether such a network is fast and reliable enough for a New Zealand general election. In particular, we test whether it can confirm votes quickly enough for a worst-case load of 100 votes per second, how its speed changes as more organisations join, and whether it records and counts every valid vote correctly even when some organisations fail or try to cheat.

2. Background and Literature Review

2.1 Voting in New Zealand Today

New Zealand's general elections are run under the Electoral Act 1993 [5]. An enrolled voter attends a voting place, has their name marked off the electoral roll, marks a paper ballot in a private booth and places it in a sealed ballot box. Voters who cannot attend their usual voting place, including those overseas, cast special votes that are processed separately.

When the poll closes, officials at each voting place count the ordinary votes by hand and report a preliminary result on election night. Over the following weeks, the marked rolls are checked to make sure no one has voted more than once, special votes are added, and every ballot paper is counted again in the official count [5]. In the 2020 and 2023 elections this took 20 days [2].

This process is thorough, but it is limited by how quickly paper can be counted and checked. It also asks voters to travel to a voting place or arrange a special vote in advance, which is harder for people in remote areas, people with disabilities and voters overseas [2, 4].

2.2 Opportunities and Risks of Online Voting

Online voting lets people cast their vote from their own device over the internet. Votes can be counted as they arrive, voters can take part from anywhere, and fewer staffed voting

places and less paper handling reduce costs [2, 4]. Estonia has offered online voting in its national elections since 2007 [2], storing encrypted votes on servers run by its election authority, which decrypts and counts them after the poll closes [6]. As with postal voting, voters and observers must trust the authority that holds the votes.

Online voting also brings new risks [2, 4]. Malware on a voter's device could change their vote, stolen credentials could let an attacker vote first, attackers could flood the system so that genuine voters cannot get through, and without a private booth a voter could be pressured or paid to vote a certain way. Most importantly for this project, a conventional online system stores votes on servers run by one organisation, so anyone who controls or breaks into those servers could alter the result, a concern also raised by an independent review of Estonia's system [6].

These risks are why New Zealand has been cautious. A planned online voting trial for the 2016 local elections did not go ahead, a proposed 2019 trial was not taken forward, and the Department of Internal Affairs has noted security concerns raised by the NZSIS and GCSB [3]. Park et al. [7] also caution that blockchain does not solve every online voting problem, particularly the security of voters' own devices. This project therefore focuses on one problem that blockchain is well suited to, which is making sure that once votes are received, no single party can change them.

2.3 Requirements for a National Online Voting System

Any voting system, paper or online, must meet a set of basic requirements before the public can trust its result. The United States Election Assistance Commission's Voluntary Voting System Guidelines [8] are a widely used reference for these, and prior work in this department grouped them into verifiability, auditability, privacy and integrity [2, 4].

For a New Zealand general election, this means only enrolled voters should be able to vote, and each vote should be counted exactly once. No single party should be able to add, change or remove votes, and it should be possible to check and re-tally the results independently. The system should also keep running if parts of it fail or come under attack, and it should confirm votes quickly, so results are available promptly. This project focuses on these requirements. They all depend on how the computers in the network agree on which votes to record, so this agreement mechanism, called consensus, is the central technical component of the project.

Privacy is the main requirement this project does not fully address. Access to the network is limited to trusted organisations, but the votes themselves are not encrypted. A voter could also show someone else proof of how they voted. This matters because the Electoral Act makes it an offence to induce a voter to show how they voted [5, s. 203], which protects voters from bribery and pressure. MACI was designed to prevent this and giving it up is the main trade-off of this project's design. It is discussed further in Section 6 and left as future work.

2.4 Estimating New Zealand Election Load

To judge whether a system is fast enough, we first need to know how much work it would face. In the 2023 general election, about 2.9 million votes were cast [1]. About 1.6 million of these were cast during the two-week advance voting period, and the busiest advance day averaged about 6.6 votes per second [9]. Prior work used this figure as New Zealand's peak voting rate [4]. However, about 1.2 million votes were cast on election day itself, between 9am and 7pm [1, 5], which is an average of roughly 34 votes per second.

Online voting is likely to raise this further, since it removes the need to travel to a voting place. In Estonia, more than half of all votes in the 2023 parliamentary election were cast online [10]. Voting also tends to cluster at certain times of day [4], so short bursts can be much higher than the daily average. To allow for this, this report uses a worst-case scenario in which every enrolled New Zealand voter, about 3.7 million people [11], votes online during election-day hours. This gives a target of about 100 votes per second.

Speed matters to two groups. Voters should know within a few seconds that their vote has been safely recorded, since long delays near the close of the poll leave them unsure whether their vote counted [4]. Election officials should be able to produce a result within a day of the poll closing, which is the target prior work set for MACI [2].

2.5 Distributed Ledgers and Blockchain

2.5.1 Why a Decentralised Distributed System

As described in Section 2.2, the main concern with a conventional online voting system is that votes are stored on servers run by one organisation. This creates two problems. If the servers go down, voting stops, and if the organisation's servers are compromised or misused, the result can be changed without anyone outside noticing.

A distributed system solves the first problem by spreading copies of the data across many servers, so that voting continues if some fail. A decentralised system solves the second problem by spreading control of the copies across several independent organisations, so that no single one can change the record. Distributed ledger technology (DLT) is both distributed and decentralised, as long as each copy is run by a different organisation and all of them follow a shared set of rules to agree on every change [2, 4].

A decentralised record can be built in several ways, and Table 1 compares the main options against the requirements in Section 2.3. A database copied across several organisations and kept in step by a crash-tolerant protocol such as Raft [12] keeps running if some copies fail, but it assumes every copy is honest, so one dishonest operator can still change records. Cryptographic voting systems such as Helios [13] and ElectionGuard [14] let voters check that their vote was recorded and let anyone check the count, but they rely on one election server to collect and publish votes, so its operator could still drop votes, which voters would have to notice and report, or take the election offline. A permissioned blockchain with a Byzantine fault tolerant protocol is the only option where no single organisation can change or lose votes and the record stays open to checking. Its weakness is privacy, which cryptographic systems handle better, so the two approaches complement each other rather than compete (Section 6.3).

Blockchain is the most widely used form of DLT, and it is the form chosen for this project for three reasons. First, a blockchain places every record in a single agreed order, which a voting system needs to decide reliably which vote came first when duplicates arrive. Second, each block is linked to the one before it, so any attempt to alter past votes is easy to detect. Third, blockchain platforms are mature and support smart contracts, which allow voting rules to be enforced by every computer in the network. Using a blockchain also keeps this project directly comparable with the MACI work, which was built on the same type of technology [2, 4].

Table 1 Comparison of approaches to recording votes.

Option	No single party can change votes	Keeps running if parts fail	Can be re-tallied independently	Vote privacy
Central server	No	Only with backups run by the same organisation	No	Depends on the operator
Copied database (Raft)	No, assumes every copy is honest	Yes, for crashes	Only by the operators	Depends on the operators
Cryptographic voting (Helios, ElectionGuard)	Changes are detectable, but votes can be dropped	No, relies on one server	Yes	Yes, votes are encrypted
Permissioned blockchain (this project)	Yes, while fewer than one-third are dishonest	Yes, up to f failures	Yes, by authorised auditors	Not yet, see Section 6.3

2.5.2 How a Blockchain Works

A blockchain is a shared record of transactions that is copied across many computers, called nodes [15]. Each node holds a full copy of the record, so no single computer owns it. New transactions, which in this project are votes, are grouped together into blocks. Each block contains a cryptographic hash of the block before it, a short fingerprint that changes completely if any part of the data changes. Altering an earlier block would break every link that follows it, so any tampering is immediately visible to the other nodes.

A new block is only added once the nodes agree that it is valid. The method they use to reach this agreement is called a consensus protocol, and it is what stops any single node from adding, changing or removing records on its own. Consensus is covered in detail in Section 2.7.

Modern blockchains can also run smart contracts, which are programs stored on the blockchain that run in the same way on every node [16]. In a voting system, a smart contract holds the voting rules, such as checking that a voter is enrolled, has not already voted and is voting while the poll is open. Every node checks each vote against these rules independently, so a vote that breaks the rules has no effect on any node's copy of the record.

2.5.3 Blocks and Block Period

A blockchain does not process each vote on its own. It collects waiting votes into a group called a block and agrees on the whole block at once. How often a new block is produced, called the block period, can be configured on a private network. A shorter block period means voters wait less for their vote to be included, but each round of agreement must finish before the next block is due. At 100 votes per second, a block produced every two seconds would hold around 200 votes.

2.5.4 Public and Permissioned Blockchains

Blockchains differ mainly in who is allowed to run a node and take part in agreement.

Public blockchains, such as Ethereum, allow anyone to join. Because participants are

anonymous, the network needs costly safeguards to stop one person from pretending to be many. This openness makes public blockchains slow for an election workload. Ethereum produces a block about every 12 seconds, takes around 12 to 15 minutes before a block can no longer be reversed, and handles only 10 to 20 transactions per second, shared between every user in the world [4]. Each transaction also costs a fee, and all data is visible to everyone, so a vote linked to a known voter would reveal how they voted [4, 7].

Permissioned blockchains limit participation to a fixed set of known organisations. Because every participant is identified, the network can use faster consensus protocols in which a block is final as soon as it is agreed. The block period can be set to suit the workload, the network's full capacity is available to the election, and access to the data can be restricted to members. The trade-off is that trust now rests on the chosen organisations. For an election, they would need to be independent of one another, so that the result stays correct as long as fewer than one-third of them are dishonest (Section 2.7).

2.6 Previous Evaluation of MACI

2.6.1 How MACI works

Minimal Anti-Collusion Infrastructure (MACI) is a voting protocol that runs as a set of smart contracts on Ethereum and compatible blockchains [17]. It was designed to solve the privacy problem of public blockchains described in Section 2.5.4. Before submitting, each voter encrypts their vote so that only a designated party, called the coordinator, can read it. Voters may also submit a new vote at any time during the poll to replace an earlier one. Because any vote could have been replaced later, a voter cannot prove to someone else how they finally voted, which protects against bribery and pressure [2, 4, 17].

After the poll closes, the coordinator decrypts and counts the votes. It then produces zero-knowledge proofs, which are mathematical proofs that the count was carried out correctly without revealing any individual vote. These proofs are checked on the blockchain, so the coordinator cannot publish a false result without being caught.

2.6.2 Performance Findings

Kwon [4] and Browne [2] evaluated MACI for a New Zealand general election in 2025 and found two main bottlenecks.

The first is vote confirmation. Kwon measured how long a vote took to be recorded on the three test networks. It took an average of 10.72 seconds on Ethereum's test network, 4.61 seconds on Gnosis Chiado and 1.27 seconds on Optimism Sepolia [4]. These delays come from the underlying public networks rather than from MACI itself. Capacity is also a concern. Ethereum handles only 10 to 20 transactions per second in total [4], which is well below the target of 100 votes per second set in Section 2.4, before any other users of the network are considered. Faster networks such as Optimism offer more capacity, but a vote only becomes permanent after a challenge period of around seven days [4].

The second bottleneck is proof generation. Browne estimated that producing the proofs for an election of five million votes would take about 2.86 weeks, or 1.27 weeks with parallel processing [2]. Many of the steps must run one after another, which limits how much they can be sped up. Against a target of publishing a verified result within a day of the poll closing, this is far too slow [2]. Both reports also noted that, because anyone can submit messages to MACI, an attacker could flood it with invalid votes that still have to be processed during counting, delaying the results further [2, 4].

2.6.3 *Implications for This Project*

These findings show that MACI does not yet meet the speed requirement from Section 2.4, for two different reasons. Slow confirmation and limited capacity come from the public network MACI runs on, while slow proof generation comes from the cryptography it uses to guarantee a correct count.

A permissioned blockchain addresses both. Running on a dedicated network removes the dependence on public infrastructure, and Kwon himself suggested that a private chain could reduce confirmation times [4]. On a permissioned blockchain, the correctness of the count is guaranteed by the consensus protocol itself, since every block of votes must be checked and approved by most of the organisations in the network. This removes the need for slow proof generation, at the cost of the privacy MACI provides, as discussed in Section 2.3.

2.7 Byzantine Fault Tolerant Consensus

2.7.1 *The Agreement Problem*

Section 2.5.2 explained that a block is only added once the nodes agree it is valid. Reaching that agreement is difficult when some nodes may be faulty. A node can fail in two ways. It can simply stop responding, which is called a crash fault, or it can behave arbitrarily, including lying or sending different messages to different nodes, which is called a Byzantine fault [18]. An election system must assume Byzantine faults, because a dishonest or compromised organisation is exactly the threat it needs to withstand.

Lamport, Shostak and Pease showed that a network can only reach reliable agreement despite f Byzantine nodes if it has at least 3f + 1 nodes in total [18]. The reasoning is as follows. A node cannot wait to hear from everyone, since up to f nodes may never reply, so it must decide once it has heard from n – f nodes. Some of those replies may themselves come from the f dishonest nodes. With at least 3f + 1 nodes, honest replies always outnumber dishonest ones. Protocols built on this result therefore require agreement from about two-thirds of the nodes, called a quorum. Any two quorums overlap in at least one honest node, so two conflicting blocks can never both be accepted at the same height.

2.7.2 *IBFT 2.0 Protocol*

Castro and Liskov's Practical Byzantine Fault Tolerance (PBFT) was the first protocol of this kind fast enough for real systems [19]. Its safety does not depend on timing, so messages can be delayed for any length of time without honest nodes ever disagreeing. Istanbul Byzantine Fault Tolerance (IBFT) adapts PBFT for blockchains [20]. Instead of agreeing on individual requests, each round of the protocol agrees on one block. The original version was found to have flaws that could, in rare cases, cause honest nodes to disagree or stop making progress. Moniz [20] published a corrected and formally proven description, and Saltini and Hyland-Wood [21] proposed IBFT 2.0, the version implemented in Hyperledger Besu [22]. Hyperledger Besu supports both IBFT 2.0 and the newer QBFT protocol, and recommends either for production networks of at least four validators, since both are Byzantine fault tolerant with immediate finality [22]. IBFT 2.0 was used in this project because its safety and liveness have been formally analysed [20, 21].

In IBFT 2.0, a fixed set of nodes called validators take turns to act as the proposer. Following the same three-phase structure as PBFT, the proposer sends a new block to every validator (propose), each validator checks it and tells every other validator that it accepts it (prepare), and once a quorum has accepted it, each validator sends its signature

to every other validator (commit) [19, 20, 22]. Once a validator holds a quorum of these signatures, the block is final and can never be reversed, unlike the 12 to 15 minute wait on Ethereum described in Section 2.5.4. Section 3.4 describes these steps in detail.

For a voting system, the prepare phase is where duplicate and invalid votes are stopped. Every validator runs every vote through the same smart contract rules, so a vote from an unenrolled voter or a second vote from the same voter has no effect on any honest validator's record. A dishonest proposer cannot make such a vote count, because if its block reports a different result from the one other validators calculate, it will not gain a quorum of prepare messages.

2.7.3 The Cost of Adding Validators

Adding validators allows more independent organisations to take part and increases how many faulty or dishonest validators the network can tolerate. However, each extra validator also adds messages to every prepare and commit phase. Table 2 shows how these grow for the network sizes tested in this project.

Table 2 Fault tolerance, quorum size and messages per block for IBFT 2.0 networks.

Validators	Faulty validators tolerated	Quorum	Prepare and commit messages per block
4	1	3	24
7	2	5	84
10	3	7	180
13	4	9	312
16	5	11	480
19	6	13	684
22	7	15	924

Going from 4 to 22 validators raises the number of faulty validators the network can survive from one to seven, but increases the number of messages per block by almost 40 times. Moniz [20] confirms that IBFT's communication grows with the square of the number of validators, while the propose phase is only a single broadcast. Seo et al. [23] note that this is why Byzantine fault tolerant protocols lose performance as more nodes are added. Agreement time is therefore expected to grow quickly as validators are added, with the prepare and commit phases taking up most of it.

This is the central trade-off of a permissioned voting network. More validators improve integrity and resilience, but each one adds to the cost of agreeing on every block. Few studies have measured this cost on Besu, since the benchmarks reviewed by E. Chang [24] kept the number of validators fixed. This project measures how that cost actually grows, and uses the results to recommend a network size for New Zealand in Section 6.1.

This is the central trade-off of a permissioned voting network. More validators improve integrity and resilience, but each one adds to the cost of agreeing on every block. This project measures how that cost actually grows, and uses the results to recommend a network size for New Zealand in Section 6.1.

2.8 Hyperledger Besu

Three widely used open-source platforms support permissioned networks run by a known set of organisations, namely Hyperledger Fabric, GoQuorum and Hyperledger Besu. They

differ in how they are built and how closely they follow Ethereum.

Hyperledger Fabric is a modular platform built specifically for business networks [25]. Its smart contracts are written in general-purpose languages such as Go or Java, and it processes transactions differently from Ethereum, running each one before ordering it rather than after. This gives Fabric a lot of flexibility, but it is not compatible with Ethereum smart contracts or tools. GoQuorum and Hyperledger Besu are both enterprise versions of Ethereum. They run the same smart contracts as Ethereum, written in the Solidity language, and support Byzantine fault tolerant consensus protocols including IBFT 2.0 [22, 26].

This project uses Hyperledger Besu. Because Besu is compatible with Ethereum, the voting contract could be written in Solidity and tested with the same Hyperledger Caliper benchmarking tool used in the MACI work [4, 27], which keeps the comparison with prior work fair. Besu also implements IBFT 2.0 [22], lets the block period and validator set be configured directly, and has built-in permissioning so that only approved organisations can join the network.

3. System Architecture and Design

3.1 System Overview

The system is made up of a network of validator nodes and a voting smart contract that runs on every node. Together, these form the backend of the voting system, which is independent of the frontend that voters use to cast their votes. Figure 1 shows how the parts fit together.

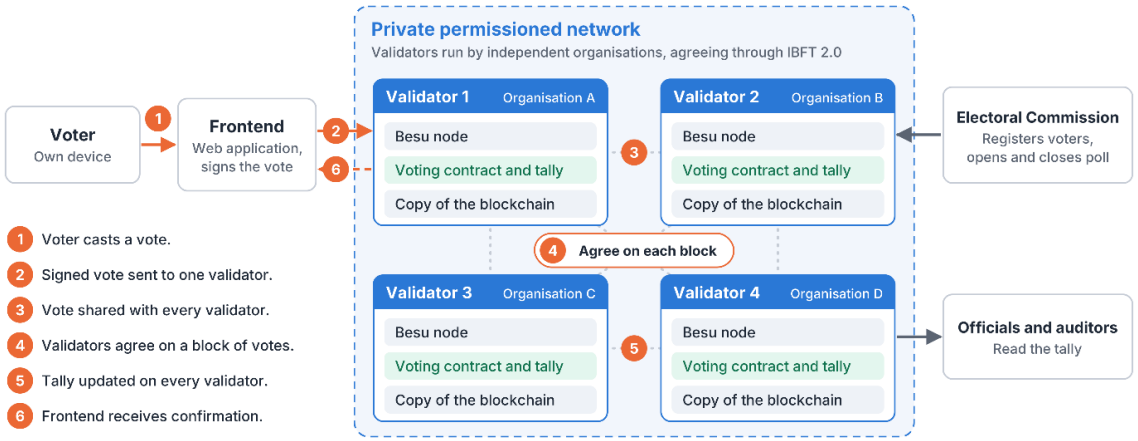


Figure 1 System architecture and the path of a vote.

Each validator node runs Hyperledger Besu and would be operated by a different independent organisation. Together, the nodes form a private permissioned blockchain that uses IBFT 2.0 to agree on each new block of votes, as described in Section 2.7. The network setup is described in Section 3.2. The voting rules are held in a smart contract, described in Section 3.3, and every validator runs the same copy of it.

When a voter casts a ballot, the frontend signs it with the voter's digital key and sends it to one of the validator nodes. From there, the vote passes through the steps shown in Figure 1. It is shared with the other validators, grouped into a block, checked against the contract rules by every validator, and added to the blockchain once a quorum agrees. Section 3.4 describes each step in detail.

In this project, a Next.js web application was used to simulate the experience from a voter's point of view, and Hyperledger Caliper and a custom load generator were used for performance testing, as described in Section 4.

3.2 Network Setup

Each validator node runs Hyperledger Besu 24.12.0 on its own computer. The nodes connect directly to each other using Besu's built-in peer-to-peer protocol, which encrypts all traffic between nodes [22]. Each node is identified by a unique cryptographic key pair, and the network only accepts connections from nodes on an approved list. An outside party therefore cannot join the network, read the traffic between nodes, or pretend to be one of the validators. E. Chang [24] describes in more detail how nodes find and connect to each other.

The network's starting state and rules are defined in a configuration file called the genesis file, which every node must share before the network starts. It lists the initial set of validators and sets the parameters that control how blocks are produced. Table 3 shows the main parameters used in this project.

Table 3 Main network parameters.

Parameter	Value	Reason
Consensus protocol	IBFT 2.0	Byzantine fault tolerant agreement with immediate finality, as discussed in Section 2.7
Block period	2 seconds	Keeps confirmation fast for voters while leaving time for each round of agreement
Round timeout	4 seconds	How long validators wait for a round to finish before starting a round change
Gas price	0	Voters do not need to pay a fee to cast a vote
Block gas limit	Set to the maximum	The number of votes in a block is limited by the network, not by an artificial cap
Storage format	Bonsai	Stores the current state efficiently, reducing disk use on each node
Transaction pool size	4,096 (Besu default)	Maximum number of votes each validator holds while waiting for a block

3.3 Voting Smart Contract

The voting rules are held in a smart contract called VotingSystem.sol, written in Solidity 0.8.19 and deployed to the network once before the election. The account that deploys it becomes the administrator, a role that would belong to the Electoral Commission in a real election. Only the administrator can enrol the voters and open or close the poll. The contract makes sure that only enrolled voters can vote and that each vote is counted exactly once, as required in Section 2.3. The contract was developed with E. Chang [24], who describes its storage and events in more detail.

When a vote arrives, the contract checks that the poll is open, the voter is enrolled, the voter has not already voted, and the chosen candidate exists. If any check fails, the vote is still included in a block as a failed transaction, but it has no effect on the tally or the

voter's status. If all four pass, the vote is added to the running tally and the voter is marked as having voted. Only the first valid vote from each voter is accepted, which makes every vote final as soon as it is recorded. The contract was tested with 10 automated Foundry tests covering enrolment, duplicate and invalid votes, and the poll window.

3.4 How a Vote is Recorded

This section follows a single vote from the moment it is submitted until it is permanently recorded. Figure 2 follows a vote through each of these steps in a network of four validators, with each colour showing the messages sent by one validator.

Submission. The frontend creates a transaction calling the contract's vote function, signs it with the voter's private key, and sends it to any one of the validator nodes. The signature proves that the vote came from the voter's account and that it has not been changed on the way.

Sharing the vote. The receiving node checks that the signature is valid and adds the vote to its transaction pool, a holding area for votes that have not yet been included in a block. It then forwards the vote to its peers, so every validator soon holds a copy and the vote does not depend on the node that first received it.

Choosing a proposer. For each new block, one validator acts as the proposer. Validators take turns in a fixed order, so every validator knows in advance who should propose the next block and no single validator controls block production.

Propose. When the block period has passed, the proposer collects pending votes from its pool, runs each one through the voting contract, and builds a block containing the votes and their results. It signs the block and broadcasts it to every other validator. In a network of n validators, this phase sends n – 1 messages.

Prepare. Each validator that receives the proposed block checks it independently. It confirms that the block came from the expected proposer and then runs every vote in the block through the contract itself. If its own results match the results in the block, it broadcasts a prepare message to every other validator. Because each of the n validators sends to the other n – 1, this phase sends about n(n – 1) messages.

Commit. Once a validator has received prepare messages from a quorum, it knows that enough validators have accepted the same block. It then broadcasts a commit message containing its signature over the block. Like the prepare phase, this sends about n(n – 1) messages.

Finalising the block. Once a validator has received commit messages from a quorum, it adds the block to its copy of the blockchain, with the quorum's signatures stored in the block header as proof of agreement. The block is now final and cannot be reversed. The voter's frontend can then retrieve a receipt showing that the vote has been recorded. The next proposer in turn then starts the next block.

The time a voter waits for confirmation therefore has two parts, the time spent waiting in the pool for the next block to start, which is up to one block period, and the time the validators take to agree on that block, which is measured in Section 5.2.

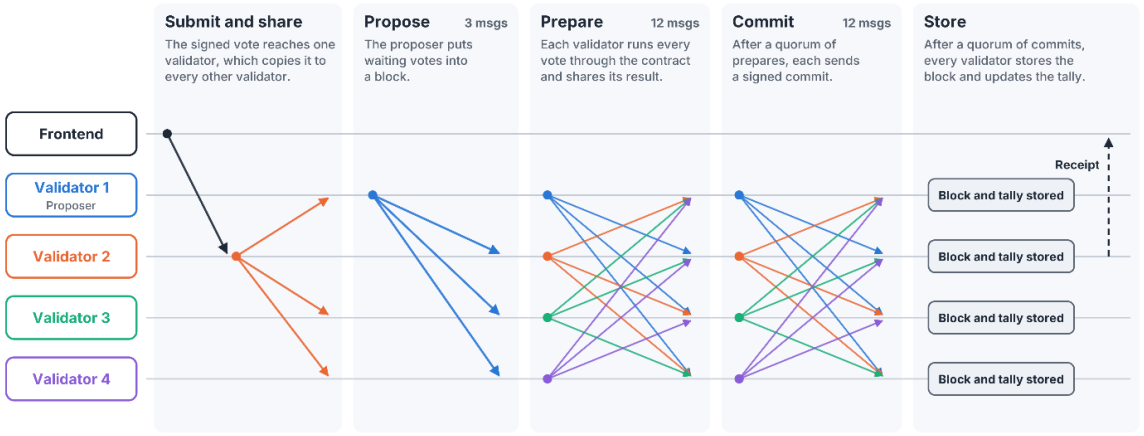


Figure 2 Path of one vote from submission to storage in a four-validator network.

3.5 How Votes are Tallied

Votes are counted as they are recorded, not after the poll closes. The voting contract keeps a running total for each candidate. When a valid vote is processed, the contract adds one to the chosen candidate's total in the same step that marks the voter as having voted, as described in Section 3.3. Both changes are recorded together in the same block or not at all. A vote can therefore never be counted without the voter being marked, and a voter can never be marked without their vote being counted.

Every validator runs every vote through the same contract, so each one calculates the tally itself and holds its own copy. The prepare phase in Section 3.4 only succeeds when validators reach the same results, so every honest validator holds the same tally after every block.

Any validator can report the current tally at any time without sending a transaction or waiting for a block. Reading it does not change the blockchain, and Section 5.4 shows that it takes only milliseconds. In a real election, the tally would only be released after the poll closes, as with paper ballots today.

When the poll closes, the administrator closes voting through the contract. Any vote processed after this has no effect, so the tally is fixed. Every validator holds its own copy of the result, so officials can read it from validators run by different organisations and confirm that they match.

The result can also be checked without relying on the contract's running totals. Each time a vote is recorded, the contract also writes a log entry, called an event, to the blockchain. An authorised auditor can read every event and count the votes again from scratch. If this recount matches the contract's totals, the result has been confirmed independently. Section 5.4 tests this and measures how long a recount would take for an election of New Zealand's size, and Section 6.3 discusses what these records mean for ballot secrecy. E. Chang [24] describes the tally and the events it relies on in more detail.

3.6 Consensus Rounds and Timing

Section 3.4 described how validators agree on a block when everything goes smoothly. A new block cannot be proposed until the 2-second block period has passed since the previous one, so the time between blocks is the longer of the block period and the time taken to agree. This section describes what happens when agreement does not go smoothly.

The round timer. When a round begins, every validator starts a timer, which in this project is set to 4 seconds. If the round has not ended with a committed block before the timer runs out, the validators assume something has gone wrong. The proposer may have crashed, sent an invalid block, or its messages may be delayed. The validator then broadcasts a round change message to every other validator. Once a validator has received round change messages from a quorum, it moves to the next round, and the next validator in the order becomes the proposer. In Hyperledger Besu, the timer doubles with each failed round at the same height, which gives a slow network more time to recover instead of repeatedly timing out [22].

A round change must not allow two different blocks to be accepted at the same height. If a quorum of validators had already prepared a block before the round failed, the new proposer must propose that same block again rather than a new one [20, 21]. This ensures that a block which may already have been accepted by some validators cannot be replaced.

Votes are never lost during a round change, as long as the validators' transaction pools have room for them (Section 5.3). They stay in the validators' pools and are included in whichever round eventually succeeds. However, each round change delays them by at least the length of the timer. A single round change in this project adds at least 4 seconds to the confirmation time of every vote waiting for that block.

Figure 3 shows how these timings fit together. In the normal case, agreement finishes well within the block period, and the next block is proposed once the 2 seconds have passed. When the proposer is offline, no block is proposed, so the other validators wait for the 4-second timer to run out before the next proposer takes over. If that proposer is also offline, the timer doubles to 8 seconds, so two offline proposers in a row delay the block by 12 seconds. In a large or slow network, agreement could also take longer than the block period or the round timer, which would cause round changes even when every validator is working correctly. The block period and round timer must therefore suit the size of the network, as discussed in Section 6.1.

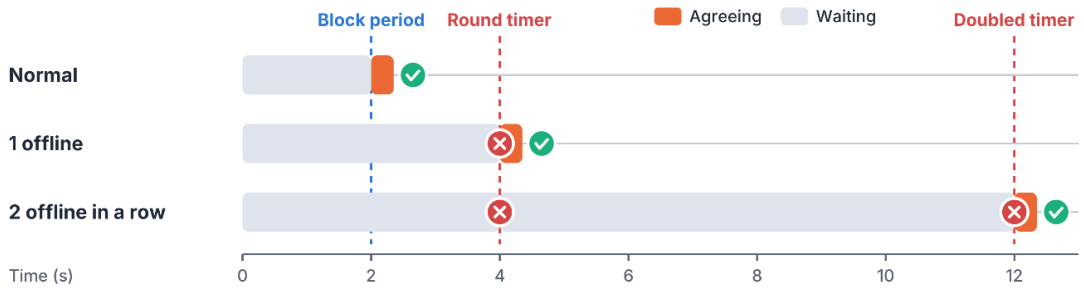


Figure 3 Timeline of one block in the normal case, with one offline proposer, and with two offline proposers in a row.

3.7 Fault Tolerance

When up to f validators crash or go offline, the remaining validators still form a quorum, so blocks continue to be agreed and votes continue to be recorded. Votes sent to a failed node are not lost, as long as the voter's frontend resends them to another node. The only effect is that when a failed validator's turn to propose comes up, no block is proposed until the round timer runs out (Section 3.6). If more than f validators fail, the rest cannot form a quorum and the network stops producing blocks. It does not accept a wrong or partial result, but simply waits until enough validators return, which in a voting system is

the safer way to fail.

A dishonest validator is handled differently from one that has simply crashed. It can misbehave in three main ways. First, it can lie to every validator in the same way, for example by proposing a block containing a vote that breaks the contract rules or a tally that does not match the votes. Every other validator runs the same votes through the contract, sees the difference and refuses to approve the block, so the round times out and the next proposer takes over. Second, it can tell different validators different things, for example approving one block to half of the network and a different block to the other half, to try to get two versions of the blockchain accepted. This cannot succeed, because any two quorums must share at least one honest validator, and that validator will only approve one of the blocks. Third, it can stay silent or leave certain votes out of the blocks it proposes. This has the same effect as a crash, delaying votes but not losing them, since the missing votes remain in every other validator's pool and are included by the next honest proposer. Attacks by dishonest validators were not tested in this project (Section 4.3.4), so this description is based on the protocol's design [20, 21].

Beyond the fault threshold, the guarantees weaken in stages. If more than one-third of validators fail or collude, they can stop the network from agreeing on new blocks, but they cannot add or change votes. If two-thirds or more collude, they can approve any block they choose, leaving out votes or producing a second, conflicting version of the blockchain. Even then, they cannot create votes for registered voters, because every vote must be signed with the voter's own key, and independent observers who keep their own copy of the blockchain would see earlier blocks being replaced. This is why the validators must be run by organisations that are independent of each other (Section 2.5.4).

When a failed validator comes back online, it first downloads the blocks it missed from the other validators. It checks each one against the commit signatures stored in its header, so it does not need to trust the node it downloads from. Once it has caught up, it takes part in agreement again as normal.

3.8 Scaling the Validator Set

A validator can be added or removed while the network is running, through a vote among the existing validators, which takes effect once more than half of them agree [22]. This allows the network to grow, or to remove a validator that has failed or can no longer be trusted, without restarting.

Every change to the validator set also changes the quorum and the number of faults the network can tolerate. Fault tolerance only increases at certain sizes. A network of four validators tolerates one fault, but so do networks of five and six. A second fault can only be tolerated once there are seven validators. Adding a fifth or sixth validator therefore makes every block more expensive to agree on without making the network any more resilient.

For this reason, this project considers networks of 4, 7, 10, 13 and larger, which are exactly the sizes of the form 3f + 1, so each step adds one more tolerated fault. Networks of up to 13 validators were tested directly, and agreement time was measured for up to 16.

4. Testing Methodology

4.1 Overview

The tests in this project were designed to answer four questions raised in Sections 1 and 2.3.

1. **Speed and capacity**. Can the network confirm votes quickly enough for a New Zealand general election, at up to 100 votes per second?
2. **Network size**. How does performance change as more validators, and therefore more independent organisations, are added?
3. **Validator failures**. Does the network keep recording votes correctly when some validators fail?
4. **Cheating and counting**. Is every valid vote counted exactly once, and can the result be checked independently?

Testing was carried out in three rounds. The first tested networks of 4, 7 and 10 validators on a new blockchain and covered all four questions. The second repeated the key tests on a blockchain holding 3.3 million registered voters and 1 million recorded votes, and measured how CPU use grows in networks of up to 16 validators. The third tested networks of 4, 7, 10 and 13 validators under light load, measured agreement time directly in networks of up to 16, and repeated the failure and attack tests at every size up to 13. The results are presented by question in Section 5.

4.2 Test Environment

In all three rounds, every validator ran as a separate Hyperledger Besu 24.12.0 process on a single virtual machine, using the network settings described in Section 3.2, with a 2-second block period and 4-second round timer. The program that generated the votes ran on the same machine. Table 4 summarises the three rounds.

Table 4 Test environment for each round.

	Round 1	**Round 2**	**Round 3**
Virtual machine	4 CPU cores, 8 GB memory	6 CPU cores, 15.6 GB memory	6 CPU cores, 15.6 GB memory
Blockchain size	New blockchain, up to about 70,000 votes per test	3.3 million registered voters, 1 million votes	New blockchain for each network size
Network sizes	4, 7 and 10 validators	4 and 7 validators under load, up to 16 with no load	4, 7, 10 and 13 validators under light load, up to 16 for agreement time
Transaction pool size	4,096 (Besu default)	65,536	65,536
Vote generator	Hyperledger Caliper 0.7.1	Custom load generator	Custom load generator, and Caliper for the failure tests

Each validator, the vote generator and a monitoring process ran as separate services with their own memory limits, so that one failing could not take down the others. The monitoring process recorded CPU and memory use, and stopped any test in which a validator crashed, stalled or stopped responding, labelling the cause. Because every validator shared one machine, the validators competed for CPU, messages between them arrived almost instantly, and the laptop host at times reduced the CPU available, so the results have limits that are discussed in Section 6.5.

4.3 Benchmarks

Every vote was sent from a different registered voter account, as it would be in a real election, and votes were spread evenly across all validators. Votes were generated with Hyperledger Caliper 0.7.1 [27], using a custom connector, since Caliper's standard Ethereum connector sends every transaction from the same account. For the longer capacity tests, a custom load generator was used instead. Caliper checks each vote's confirmation separately, and in Besu 24.12.0 each check re-reads the whole block, so at higher rates this extra work alone stalled the network. The custom generator sends votes at a fixed rate and reads each new block only once.

The following measurements are used throughout Section 5.

- **Confirmed throughput**. The number of votes per second that were included in committed blocks.
- **Confirmation latency**. The time each vote took to be confirmed, from the moment it was submitted to the moment it appeared in a committed block, reported as the median (p50) and the 95th and 99th percentiles (p95, p99). The percentiles show how long the slowest voters waited.
- **Failed votes**. Votes that were submitted but never recorded.
- **Block gap**. The time between consecutive blocks, which shows when the network falls behind its 2-second block period.
- **Recovery time**. How long a restarted validator took to catch up, and how long the network took to resume after it stopped.

A rate was counted as sustainable only if at least 99.9% of votes succeeded, the p95 latency stayed under 6 seconds, the transaction pool did not keep growing, and blocks were produced every 2.0 to 2.2 seconds.

4.3.1 Speed and Capacity

The network was sent votes at increasing rates, from 10 up to 250 votes per second, until it could no longer keep up, to find the highest rate it could sustain. This was done on a new blockchain and on a blockchain holding 3.3 million registered voters, close to New Zealand's 3.7 million enrolled voters [11], to check whether a full electoral roll reduces capacity. A total of 1 million votes was then cast on the election-scale blockchain, to check that the network kept recording votes correctly over a long period. The 4-validator network was also given two further tests.

- **Spike**. 30 votes per second for 2 minutes, a sudden jump to 150 for 1 minute, then back to 30 for 3 minutes, to check how the network recovers from a sudden rush.

- **Stress**. A rate starting at 20 votes per second and rising by 20 every 20 seconds up to 200, to find where latency starts to climb sharply.

4.3.2 Network Size

As shown in Table 2, every validator added increases the number of messages needed to agree on each block. Three measurements were used to show how this affects the network.

- **Agreement time.** How long the validators take to agree on each block. The block period was reduced to 100 ms using a test-only Besu setting, so that each new block started almost as soon as the previous one had been agreed, and the time between blocks minus the 100 ms gives the agreement time. This was measured at each network size from 4 to 16 validators, with no votes being sent and at 10 votes per second.
- **Processing cost.** The CPU used by the whole network with no votes being sent. IBFT still agrees on an empty block every 2 seconds, so this CPU use comes almost entirely from consensus. This was measured at each network size from 4 to 16 validators.
- **Confirmation latency.** How long voters wait at each network size, measured by sending the same rates of votes to networks of 4, 7, 10 and 13 validators.

4.3.3 Validator Failures

These tests simulate validators losing power during an election. Validators were shut down abruptly, without any warning, while votes continued to be sent to the validators that remained running. Two scenarios were tested.

- **Within tolerance**. f validators were shut down and later restarted. The test checked that blocks continued to be produced, measured how long the restarted validators took to catch up, and confirmed that every validator ended with an identical copy of the blockchain.
- **Over tolerance**. f + 1 validators were shut down and later restarted. The test checked that the network stopped producing blocks without splitting into conflicting chains, and measured how long it took to resume once enough validators had returned.

In every case, the offline validators were next to each other in the proposer order, which is the worst case for delays. The over-tolerance test was run with Besu's default transaction pool of 4,096 and with a larger pool of 65,536, to see whether votes sent while the network had stopped were kept.

4.3.4 Cheating and Counting

Three attacks by voters were tested.

- **Double vote**. A voter who has already voted sends a second vote.
- **Equivocation**. A voter sends two different votes at the same moment to two different validators, hoping both are counted.

- **Replay**. An attacker resends an exact copy of a vote that has already been recorded.

Each attack was attempted repeatedly on networks of 4 to 13 validators, including while the network was recording votes from other voters. In addition, 1,000 voters who had already voted were each sent a second vote for a different candidate. After every attempt, every validator was checked to confirm that it reported the same tally and the same latest block. Attacks by a dishonest validator, such as sending false or conflicting consensus messages, were not tested, so the behaviour described for them in Section 3.7 is based on the protocol design rather than measurement.

The tally was also checked independently, as an auditor would check it in a real election. In a tally audit carried out by E. Chang [24], test elections of 10,000 votes were run with and without a validator being shut down and restarted during voting, and the tally on each validator was compared with a separate recount of every vote event, as described in Section 3.5. The time taken to recount and to read the tally was also measured. At the end of the 1 million vote test, every one of the 3.3 million voter records was checked. Because each test voter's choice was known in advance, the expected tally could be calculated from the voters marked as having voted, and compared with the tally held by each validator.

5. Results and Analysis

5.1 Speed and Capacity

Figure 4 shows the results of the round 1 capacity test described in Section 4.3.1. Points on the dashed line in Figure 4(a) confirmed every vote sent. No vote failed in any completed step, so every slowdown described below shows up as extra waiting time rather than lost votes.

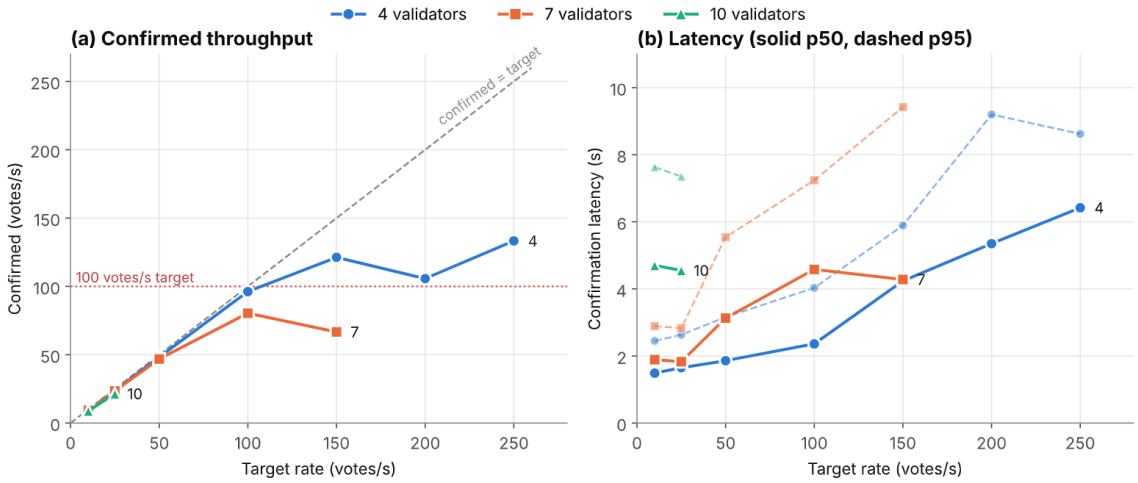


Figure 4 Confirmed throughput (a) and confirmation latency (b) at each target rate for networks of 4, 7 and 10 validators.

The 4-validator network kept up with every rate up to 100 votes per second, confirming 96.2 votes per second at that target with a median latency of 2.36 seconds. It reached its limit at around 120 to 135 votes per second, but blocks were still produced every 2 seconds, so the network did not fall behind. At 10 votes per second, it confirmed votes in an average of 1.47 seconds. That is about seven times faster than the 10.72 seconds Kwon measured for MACI on Ethereum's test network [4], and every vote is final as soon as it is confirmed.

Figure 5 shows how the 4-validator network responded to the spike and stress tests. Every vote in both tests was recorded.

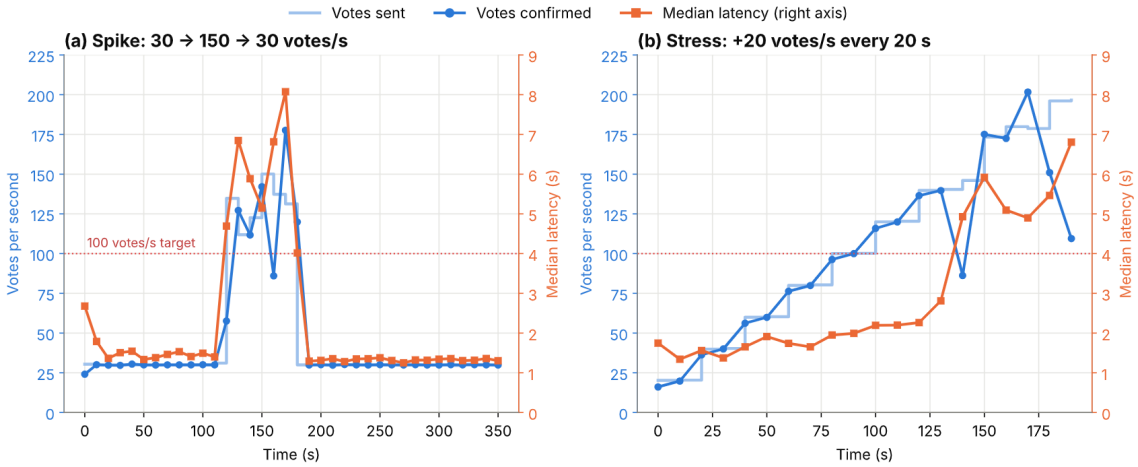


Figure 5 Votes sent, votes confirmed and median latency (right axis) over time on the 4-validator network during (a) the spike test and (b) the stress test.

In the stress test, confirmed votes kept pace with votes sent up to 140 votes per second, with a median latency between 1.5 and 2.5 seconds. Above this, latency rose sharply to more than 5 seconds. In the spike test, median latency rose from about 1.5 seconds to between 5 and 8 seconds while the rate was at 150 votes per second, but blocks were still produced every 2 seconds. Once the rate dropped back to 30, the network cleared the queued votes within about 10 seconds. A short rush beyond the network's capacity therefore causes a temporary delay rather than lost votes.

In round 2, with 3.3 million voters registered, the 4-validator network sustained 150 votes per second with a p95 latency of 3.0 seconds, the same limit it reached on a new blockchain on the same machine. The size of the electoral roll therefore did not reduce the network's capacity.

Across the 1 million votes that followed, every vote sent was recorded, with none failed or lost. However, the rate had to be lowered from 150 votes per second to 120 and then 90, and the p95 latency stayed under 6 seconds in only one of the five rounds, rising as high as 35 seconds, mostly when the laptop host reduced the CPU available to the virtual machine (Section 6.5). The network also stopped twice because of faults in Besu itself (Section 5.3). Steady performance over a full election therefore needs dedicated hardware, but even under these conditions the network never lost or miscounted a vote.

Overall, the network comfortably exceeds the target of 100 votes per second, both on a new blockchain and with a full electoral roll.

5.2 Network Size

Figure 6 shows how the cost of consensus grows as validators are added, measured as agreement time and processing cost.

Figure 6(a) shows the agreement time measured in round 3. With no votes being sent, the validators took a median of 16 ms to agree on a block with four validators, 77 ms with seven, 122 ms with ten, 223 ms with thirteen and 558 ms with sixteen. Even with sixteen validators, agreement took about a quarter of the 2-second block period, so at every size tested it was the block period, not agreement, that set the time between blocks. A light

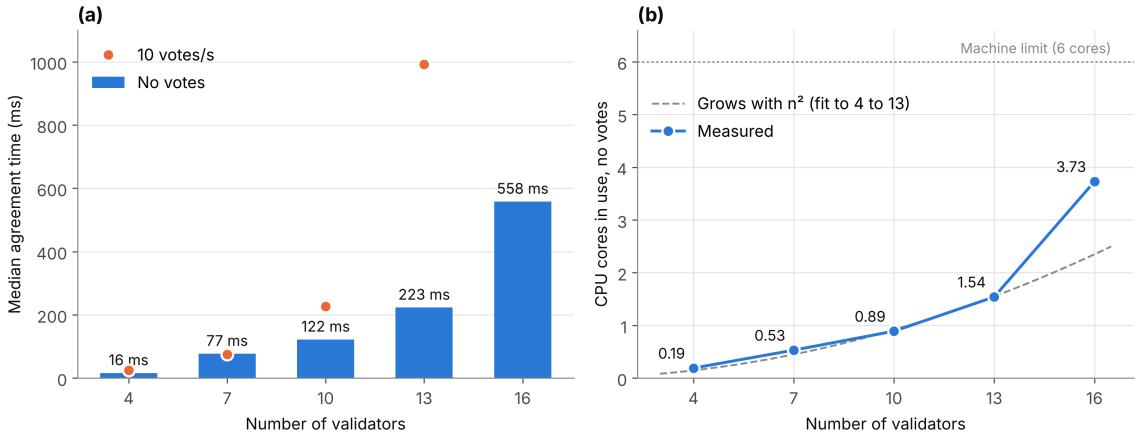


Figure 6 Measured agreement time per block (a) and CPU cores in use with no votes being sent (b), by number of validators.

load of 10 votes per second made no difference with four or seven validators, but added about 0.1 seconds with ten and 0.8 seconds with thirteen, where it pushed the machine's CPU to 99%.

Figure 6(b) shows the processing cost measured in round 2. With no votes being sent, the validators together kept 0.19 of the machine's six CPU cores busy with four validators, 0.53 with seven, 0.89 with ten and 1.54 with thirteen. These values closely follow a curve proportional to the square of the network size (R^2 = 0.996), much more closely than a straight line (R^2 = 0.971). Each validator's own CPU use grows in proportion to the number of other validators it must exchange messages with, so the total grows with the square, as Table 2 predicts. With sixteen validators, CPU use reached 3.73 cores, well above the curve, because the machine was close to its limit even with no votes being sent and blocks slowed to one every 2.24 seconds.

In round 3, confirmation latency was measured at each recommended size, with the normal 2-second block period. Networks of 4 and 7 validators sustained 50 votes per second, the highest rate tested, with a p95 latency of about 3 seconds or less. Ten validators sustained 25 votes per second with a p95 of 3.9 seconds, and thirteen sustained 10 votes per second with a p95 of 3.5 seconds, but ran out of memory at 25 votes per second, when the validators together used about 13 GB of the machine's 15.6 GB. At 10 votes per second, the median latency rose only from 1.3 seconds with four validators to 2.1 seconds with thirteen. Seven validators did not sustain 50 votes per second in round 2 with the same settings, so this limit depends on how busy the shared machine is.

Figure 4 shows the same pattern in the round 1 capacity test. The 7-validator network reached its limit at around 80 votes per second, and the 10-validator network ran out of memory at 50 votes per second. Even at 10 votes per second, median latency rose from 1.49 seconds with four validators to 1.89 seconds with seven and 4.70 seconds with ten, much of it because the shared machine had only four CPU cores.

Together, these results show that agreement stays fast as validators are added, even though its cost grows with the square of the network size, and that on a single shared machine the CPU and memory run out long before the protocol does. On separate machines this limit is removed, but messages between validators take longer to arrive, which was not measured in this project (Section 6.5).

5.3 Validator Failures

Figure 7 shows the longest gap between blocks while f validators were offline in round 3, at each network size.

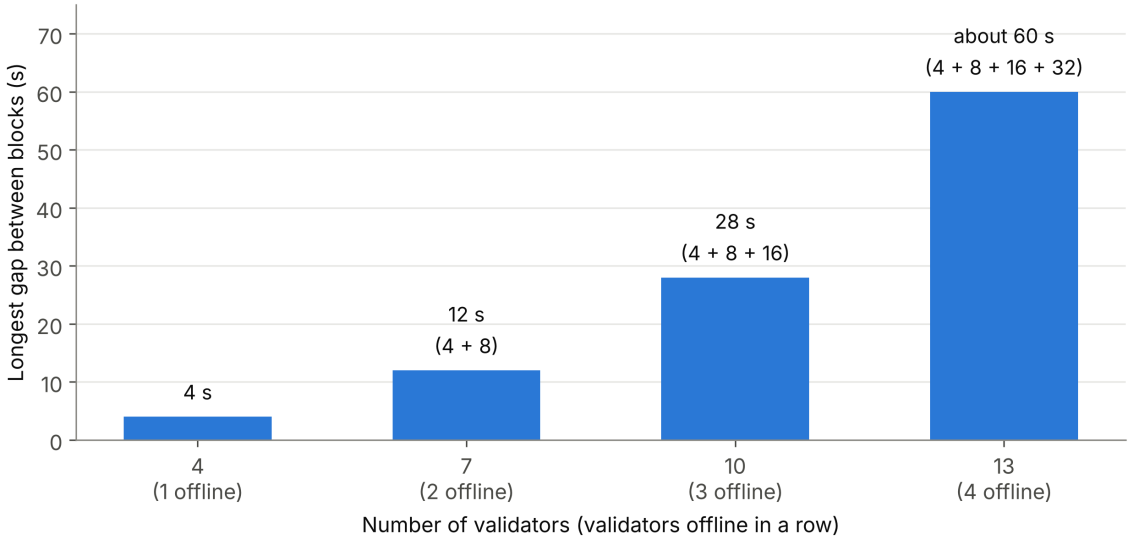


Figure 7 Longest gap between blocks while f validators were offline, by number of validators.

In round 1, with f validators offline, the 4- and 7-validator networks kept recording votes at the full rate, and the median latency rose only slightly. The p95 latency rose to 4.2 seconds with four validators and 13.5 seconds with seven, where the two offline validators were next to each other in the proposer order, so each stall lasted 4 + 8 = 12 seconds. The offline validators caught up within 5 to 11 seconds of restarting, and every validator ended with an identical copy of the blockchain.

In round 2, on the election-scale blockchain, with one of four validators offline for 5 minutes, the network kept recording votes with none failing, and the p95 latency rose only from 4.0 to 5.2 seconds. The restarted validator took 48 seconds to catch up.

In round 3, with 1, 2, 3 and 4 validators offline in networks of 4, 7, 10 and 13, the network kept producing blocks, no vote was lost, the restarted validators caught up within 1 to 9 seconds, and every validator ended with an identical copy of the blockchain. Because the offline validators were next to each other in the proposer order, each additional one doubled the wait. The longest gap between blocks was 4 seconds with one validator offline, 12 seconds (4 + 8) with two, 28 seconds (4 + 8 + 16) with three and about 60 seconds (4 + 8 + 16 + 32) with four. The median latency barely changed, so only the slowest votes were delayed.

During the 13-validator test, the laptop also paused the virtual machine for about 3 minutes, which stopped every process, including the load generator. As a result, 281 votes took longer than the load generator's 2-minute limit to be confirmed, but all 2,600 votes were still recorded on the blockchain.

With f + 1 validators offline, the network stopped producing blocks in every run of both rounds, and no conflicting chains were created. Recovery was slow. While the network was stalled, each validator kept doubling its round timer, up to 256 seconds, and blocks only resumed once enough validators happened to reach the same round. Votes submitted during the stall wait in each validator's transaction pool, and any backlog beyond the pool's

limit is discarded. Table 5 compares the two rounds.

Table 5 Over-tolerance results for the 4-validator network.

	Round 1	**Round 2**
Validators offline	2 of 4, for 60 seconds	2 of 4, for 3 minutes
Transaction pool size	4,096	65,536
Conflicting chains	None	None
Votes lost during the stall	2,340 of 7,503 (31%)	0 of 5,406
Time to resume after restart	135 to 266 seconds	286 seconds

With Besu's default pool of 4,096, around a third of the votes sent during the stall were silently lost. With the pool raised to 65,536, every vote sent during the stall was recorded once the network resumed. The pool size therefore decides whether an outage delays votes or loses them.

The network can also stop for reasons other than validators failing. In round 2, under sustained heavy load, Besu 24.12.0 twice stopped producing blocks, once when a validator's consensus process became stuck while a large block was being imported, and once when two validators that had caught up through Besu's syncing process stopped taking part in agreement. With four validators, either problem stopped the network until the affected validators were restarted. In round 3, the same lock-up occurred once with ten validators and once with thirteen, before any validator had been shut down, and both tests passed when repeated. In every case the network stopped safely, with no conflicting chains and no lost or altered votes.

Across all three rounds, a failure never caused a vote to be recorded incorrectly. What failures affect is how long voting is unavailable, which depends on the transaction pool size, the proposer order and how quickly stuck validators are detected and restarted.

5.4 Cheating and Counting

All 27 attack attempts in round 1 were stopped, three for each attack on networks of 4, 7 and 10 validators, and after every attempt all validators reported the same tally and the same latest block. Each attack was stopped in a different way. A double vote was included in a block, but the voting contract saw that the voter had already voted, so the vote had no effect and the tally increased by exactly one. In the equivocation attack, both conflicting votes reached the network, but exactly one was recorded every time. Which one won varied between attempts, showing a genuine race rather than one validator always being favoured, and because every validator follows the same block order, they all agreed on which vote counted. A replayed vote was refused as soon as it was submitted, since every transaction from an account carries a sequence number, called a nonce, that can only be used once.

These results show that the two layers of the system protect against different attacks. The voting contract stops a voter from voting twice, while the blockchain itself, through its single agreed order and one-time sequence numbers, stops conflicting or copied votes from being counted.

In round 2, the 27 attacks were repeated on the 4-validator network after the 1 million votes had been recorded, with the first eight sent while the network was recording 90 votes per second from other voters. All 27 were stopped in the same ways. In the mass double-vote test, all 1,000 second votes had no effect, and the tallies were the same before and after.

In round 3, each attack was attempted 12 times on a 13-validator network, and all 36 were stopped in the same ways. After every attempt, all 13 validators reported the same tally and the same latest block.

The tally was also checked independently. In E. Chang's tally audit [24], the tally on every validator matched a separate recount of every vote event in all six runs, including on the validator that had crashed and restarted. The recount took about 2 seconds per 10,000 votes on each validator, which suggests a full recount of a 2.9 million vote election would take about 9 to 10 minutes per validator. Reading the tally also stayed fast, with 99% of reads completing within 21 milliseconds after 10,000 votes.

At the end of round 2, the whole test election was checked. All four validators reported the same latest block, and the tally held by every validator matched exactly the tally calculated from every voter marked as having voted, 1,205,722 votes in total. Across the entire test, no vote was lost, counted twice or changed, so the tally kept by the network can be trusted and re-tallied independently from the blockchain.

6. Discussion

6.1 Recommended Network Size

Each step of three validators lets the network tolerate one more failed or dishonest validator. Section 5.2 showed that this costs little in agreement time, since even sixteen validators agreed on a block in about half a second. The cost of a larger network appears instead when several validators fail at once. Section 5.3 showed that each offline validator next in the proposer order doubles the wait, so a network that can survive more failures also faces longer delays when those failures happen together.

With the 2-second block period and 4-second round timer used in testing, agreement fits well within each block at every size measured, so both settings suit networks of up to 16 validators. Table 6 summarises the measured results for each network size.

Table 6 Measured results for each network size.

Validators	Faults tolerated	Agreement time (s)	Median voter wait (s)	Longest delay with f validators offline in a row (s)
4	1	0.02	1.3	4
7	2	0.08	1.3	12
10	3	0.12	1.7	28
13	4	0.22	2.1	60
16	5	0.56	Not measured	Not measured

Based on this, a network of 10 to 13 validators is recommended for a New Zealand general election. Ten validators can survive three failed or dishonest validators and thirteen can survive four, while a typical voter still waits about 2 seconds with the normal 2-second block period. The longer delays in Table 6 only occur when validators next to each other in the proposer order fail together, so validators likely to fail together, such as those run by the same organisation or hosted in the same data centre, should be spread out in the proposer order. Failed validators should also be detected and restarted quickly, so that they do not stay offline long enough to delay many votes (Section 6.4).

These recommendations are based on measurements on a single shared machine with no network delay. On separate machines, each validator would have a machine to itself, removing the CPU and memory limits seen in testing, but messages between validators would take longer to arrive. Since each block needs three rounds of messages, validators within New Zealand, where messages typically arrive within tens of milliseconds, should still agree well within the block period. This should be confirmed by testing on separate machines before the recommendations are relied on, as discussed in Section 6.5.

6.2 Comparison with MACI

Table 7 compares the results of this project with the MACI results summarised in Section 2.6.

Table 7 Comparison with the MACI evaluation.

	MACI on public networks [2,4]	**This project (4 validators)**
Average confirmation time	10.72 s on Ethereum, 1.27 s on Optimism	1.47 s
Time until a vote is final	12 to 15 minutes on Ethereum, about 7 days on Optimism	Immediate
Capacity	10 to 20 transactions per second on Ethereum, shared with all users	About 150 votes per second on one shared virtual machine, including with 3.3 million voters registered
Time to a verified result	1.27 to 2.86 weeks of proof generation for 5 million votes	Running tally kept on every validator (Section 3.5). About 10 minutes for an independent recount of 2.9 million votes
Vote privacy	Votes encrypted, and voters cannot prove how they voted	Votes visible to anyone who can read the blockchain

On every measure of speed, the permissioned network performs far better. Votes are confirmed about seven times faster than on Ethereum and as fast as on Optimism, while being final immediately rather than after minutes or days. Most importantly, the slowest part of MACI, generating the proofs that certify the results, is not needed at all. The tally is updated as each vote is recorded and can be independently recounted from the blockchain in minutes, so a verified result could be available on election night, as the paper count currently provides.

These gains come from two differences in design. The network is dedicated to the election rather than shared with every other user of a public blockchain, which removes the delays and capacity limits Kwon measured [4]. The count is also guaranteed by the consensus protocol, since every validator checks every vote, rather than by cryptographic proofs produced after the poll closes, which removes the bottleneck Browne measured [2].

The comparison is not entirely like for like, since the MACI results were measured on public networks across the internet and this project's on a single machine. However, Section 5.2 showed that the consensus cost itself is small, and network delay between New Zealand locations should add only a fraction of a second to each block.

The main difference is privacy. MACI encrypts every vote and lets voters replace an earlier vote, so no one can prove how they voted. This project does not encrypt votes, and each recorded vote can be linked to the voter's account, as discussed in Section 6.3.

6.3 Ballot Secrecy

The network keeps votes from the outside world, since only approved validators can join and all traffic between them is encrypted. Inside the network, however, every recorded vote stores the voter's account address alongside the chosen candidate. The address alone does not identify the voter, since the link to a real person is held by whoever enrolled the voter, which in a real election would be the Electoral Commission. This is much like a paper ballot number, which only identifies a voter when officials match it against the roll under specific legal conditions [5, ss. 167, 176]. On the blockchain, though, anyone with access to a validator and the enrolment list could link every vote to a voter.

A voter could also prove how they voted by showing the transaction that recorded it, which the Electoral Act aims to prevent [5, s. 203]. Both issues would need to be addressed before a real election, and two approaches could help.

- **Encrypting votes.** Each vote would be encrypted before it is sent, so validators can confirm that a voter has voted without being able to read their choice. Votes would only be decrypted for counting after the poll closes. This would replace the running tally described in Section 3.5 with a single count at the end.
- **Separating voters from voting accounts.** Each enrolled voter would be given a one-time voting account that is not recorded against their name, so a recorded vote cannot be traced back to a person.

Either approach would reduce what validators can learn, but neither on its own would stop a voter proving how they voted. Fully restoring that protection would require techniques such as those used in MACI, and exploring how much privacy can be added without losing the speed demonstrated here is the most important direction for future work (Section 7). E. Chang [24] discusses these options further.

6.4 Lessons for Deployment

The tests also revealed practical issues that a real deployment would need to handle. None affect the correctness of recorded votes, but each affects how reliably the network serves voters.

Design constraints

- **Recovering from a lost quorum**. Section 5.3 showed that after too many validators went offline, the network took three to five minutes to resume, even after the validators were back. This is because each validator keeps doubling its round timer while the network is stalled, and blocks only resume once enough of them reach the same round.
- **Enrolment key.** Only the account that deployed the voting contract can enrol voters (Section 3.3), so whoever holds its private key could register voters who do not exist. E. Chang [24] suggests splitting this key between several organisations, so that no single one can change the voter roll.
- **Validator order.** Validators likely to fail together, such as those run by the same organisation or hosted in the same data centre, should be spread out in the proposer order (Section 6.1).

- **Confirming each vote.** When a vote is submitted, the network returns a transaction hash straight away, before the vote is recorded. A frontend should not treat this as confirmation. It should wait until the vote appears in a block and resend it if it does not. Resending is safe, because the replay test in Section 5.4 showed that a vote already recorded cannot be counted twice.

- **In-person voting.** Online voting would sit alongside in-person voting rather than replace it, since not every voter can or will vote online. What makes paper voting trustworthy is that officials confirm each voter's identity and the vote is cast privately in a booth. Digital kiosks at voting places would keep both, while recording votes on the same blockchain as online votes. Because both kinds of vote pass through the same contract, a voter who has voted online cannot vote again at a kiosk, and the reverse. If each kiosk signs votes with a one-time key that the voter never holds, the voter also cannot prove how they voted, which addresses part of the privacy gap described in Section 6.3. Paper ballots would remain as a fallback if a voting place loses its connection to the network.

Hardware and software constraints

- **Transaction pool size.** Votes sent while the network is stalled are held in each validator's transaction pool, and anything beyond its limit is discarded. The pool should be sized for the largest backlog expected, roughly the peak voting rate multiplied by the longest outage the network should survive. Raising it from Besu's default of 4,096 to 65,536 kept every vote in the outage tested, where the default lost 31%.

- **Monitoring validators.** Section 5.3 showed that Besu 24.12.0 can stop, both under heavy load and at light load, when a validator locks up or stops taking part in agreement after catching up. Each validator should be monitored so that one that stops participating is detected and restarted quickly, and newer Besu releases should be tested for these faults before deployment.

- **Storage.** Every vote is stored permanently on every validator. E. Chang [24] measured about 3.5 kB per vote, an estimated 15.7 GB per validator for a national election, which is well within the capacity of a typical server.

In general, the system can address these issues without any changes to the consensus protocol or the voting contract. The design constraints can be managed through how validators are run and how the frontend handles votes, while the hardware and software constraints can be met with appropriate configuration and more capable machines than those used in testing.

6.5 Limitations

All tests in this project were run in a controlled environment on a single machine. This made it possible to test many network sizes and failure scenarios repeatably, but it also means the conditions differ from a real election network in two important ways.

- **Shared hardware.** Every validator and the load generator ran on a single virtual machine with 4 CPU cores in round 1 and 6 in rounds 2 and 3, only part of the host computer's capacity. The throughput limits measured are therefore set by the test

machine rather than the system, and the slowdown measured at larger network sizes overstates the cost of consensus. In round 2, the laptop hosting the virtual machine also reduced its CPU at times, which is why the long run missed its latency targets. With sixteen validators, the machine was almost fully busy even with no votes being sent, so the agreement time measured at that size is an upper bound.

- **No network delay.** Because every validator ran on the same machine, messages between them arrived almost instantly. Validators in different locations would add network delay to every phase of agreement. This is the main cost that a larger network would add in a real deployment (Section 6.1).

Further testing that would extend these results, including larger networks on separate machines and attacks by dishonest validators, is discussed in Section 7.

7. Conclusions and Future Work

The project evaluated whether a private blockchain using Hyperledger Besu and IBFT 2.0 could support a New Zealand general election. A 4-validator network confirmed votes in about 1.5 seconds and sustained 150 votes per second on a single shared machine, including with 3.3 million voters registered, above the worst-case target of 100 votes per second. Adding validators barely slowed agreement, which took about 0.2 seconds with thirteen validators and half a second with sixteen. Compared with MACI, votes were confirmed about seven times faster than on Ethereum, were final immediately, and the tally could be independently recounted in minutes rather than weeks.

The network also behaved correctly under failure. With up to f validators offline it kept recording every vote at every network size from 4 to 13, and with more offline it stopped safely rather than accepting a wrong result. Every attempt by a voter to cheat was stopped, and independent recounts matched the recorded tally on every validator. In a test election of 1 million votes, no vote was lost, counted twice or changed. Adding validators improves this resilience at little cost to speed, but makes delays longer when several validators fail together, and a network of 10 to 13 validators is recommended as a balance between the two.

These results show that a private blockchain is a viable option for New Zealand elections, but it would need further testing and careful design before it could be put into use. Running each validator on its own machine, in different locations, would show how much network delay adds to agreement and whether 10 to 13 validators can sustain 100 votes per second, and would allow attacks by dishonest validators to be tested. Most importantly, votes would need to be encrypted or separated from voter identities to match the privacy of paper voting, and identity checks such as RealMe would need to be integrated before voters could use the system. Public trust would also need to be earned, for example by publishing the voting contract's source code, giving independent observers the full blockchain to recount once voting closes, and trialling the system in smaller elections first. During voting, observers could be given only the fingerprint of each block, so that they can later confirm no block was changed without seeing any votes before the poll closes.

References

[1] Electoral Commission of New Zealand, "2023 general election – official results and statistics," 2023. [Online]. Available: https://archive.electionresults.govt.nz/electionresults_2023/

[2] J. Browne, "Evaluating the scalability and suitability of MACI as an online solution for New Zealand's general election [Unpublished project report, University of Auckland]," 2025.

[3] Department of Internal Affairs, "Online voting," dia.govt.nz, accessed October 13, 2026. [Online]. Available: https://www.dia.govt.nz/online-voting

[4] C. Kwon, "Evaluating the scalability and suitability of MACI as an online solution for New Zealand's general election [Unpublished project report, University of Auckland]," 2025.

[5] "Electoral Act 1993 (New Zealand)," Version as at March 31, 2023. [Online]. Available: https://www.legislation.govt.nz

[6] D. Springall, T. Finkenauer, Z. Durumeric, J. Kitcat, H. Hursti, M. MacAlpine, and J. A. Halderman, "Security analysis of the Estonian internet voting system," in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*, Scottsdale, AZ, USA, 2014, pp. 703–715.

[7] S. Park, M. Specter, N. Narula, and R. L. Rivest, "Going from bad to worse: From internet voting to blockchain voting," *Journal of Cybersecurity*, vol. 7, no. 1, p. tyaa025, 2021.

[8] U.S. Election Assistance Commission, "Voluntary voting system guidelines version 2.0," Washington, DC, USA, 2021. [Online]. Available: https://www.eac.gov/voting-equipment/voluntary-voting-system-guidelines

[9] Electoral Commission, "2023 general election: Voting statistics," elections.nz, accessed October 17, 2026. [Online]. Available: https://elections.nz/democracy-in-nz/historical-events/2023-general-election/voting-statistics

[10] State Electoral Office of Estonia, "Statistics about internet voting in Estonia," valimised.ee, accessed October 13, 2026. [Online]. Available: https://www.valimised.ee/en/archive/statistics-about-internet-voting-estonia

[11] Electoral Commission, "2023 general election: Voter turnout statistics," elections.nz, accessed October 18, 2026. [Online]. Available: https://elections.nz/democracy-in-nz/historical-events/2023-general-election/voter-turnout-statistics

[12] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *Proceedings of the USENIX Annual Technical Conference*, Philadelphia, PA, USA, 2014, pp. 305–319.

[13] B. Adida, "Helios: Web-based open-audit voting," in *Proceedings of the 17th USENIX Security Symposium*, San Jose, CA, USA, 2008, pp. 335–348.

[14] J. Benaloh, M. Naehrig, O. Pereira, and D. S. Wallach, "ElectionGuard: A cryptographic toolkit to enable verifiable elections," in *Proceedings of the 33rd USENIX Security Symposium*, Philadelphia, PA, USA, 2024.

[15] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[16] V. Buterin, "Ethereum: A next-generation smart contract and decentralized application platform," 2014. [Online]. Available: https://ethereum.org/whitepaper

[17] Privacy and Scaling Explorations, "MACI documentation," Ethereum Foundation, accessed October 13, 2026. [Online]. Available: https://maci.pse.dev/

[18] L. Lamport, R. Shostak, and M. Pease, "The Byzantine generals problem," *ACM Transactions on Programming Languages and Systems*, vol. 4, no. 3, pp. 382–401, 1982.

[19] M. Castro and B. Liskov, "Practical Byzantine fault tolerance," in *Proceedings of the 3rd Symposium on Operating Systems Design and Implementation*, New Orleans, LA, USA, 1999, pp. 173–186.

[20] H. Moniz, "The Istanbul BFT consensus algorithm," arXiv preprint arXiv:2002.03613, 2020.

[21] R. Saltini and D. Hyland-Wood, "IBFT 2.0: A safe and live variation of the IBFT blockchain consensus protocol for eventually synchronous networks," arXiv preprint arXiv:1909.10194, 2019.

[22] Hyperledger Besu, "IBFT 2.0," Besu documentation, accessed October 13, 2026. [Online]. Available: https://besu.hyperledger.org

[23] J. Seo, D. Ko, S. Kim, and S. Park, "A coordination technique for improving scalability of Byzantine fault-tolerant consensus," *Applied Sciences*, vol. 10, no. 21, p. 7609, 2020.

[24] E. Chang, "Assessing the suitability of blockchain frameworks for New Zealand general elections [Unpublished project report, University of Auckland]," 2026.

[25] E. Androulaki *et al.*, "Hyperledger Fabric: A distributed operating system for permissioned blockchains," in *Proceedings of the 13th EuroSys Conference*, Porto, Portugal, 2018.

[26] ConsenSys, "GoQuorum documentation," accessed October 17, 2026. [Online]. Available: https://docs.goquorum.consensys.io

[27] Hyperledger Foundation, "Hyperledger Caliper documentation," accessed October 17, 2026. [Online]. Available: https://hyperledger-caliper.github.io/caliper/